

Introducción a la Programación I

Clase 7: Tuplas y Conjuntos (Sets)

¿Qué es una Tupla?

Una **tupla** es una colección **ordenada** e **inmutable** de elementos.

- **Ordenada:** los elementos tienen un orden fijo, con índices (0, 1, 2...)
- **Inmutable:** una vez creada, **no se puede modificar**

```
# Lista (mutable)
lista = [1, 2, 3]
lista[0] = 99    #  permitido

# Tupla (inmutable)
tupla = (1, 2, 3)
tupla[0] = 99    #  TypeError!
```

Las tuplas son ideales para datos que **no deben cambiar**.

Crear tuplas en Python

Las tuplas se definen con **paréntesis ()**:

```
# Tupla vacía
vacía = ()

# Tupla de frutas
frutas = ("manzana", "banana", "naranja")

# Coordenadas
coordenadas = (40.7128, -74.0060)

# Tupla mixta
datos = ("Juan", 25, True, 1.75)

# Tupla de un solo elemento (¡ojo con la coma!)
uno = (42,) # ← la coma es obligatoria
```

Acceder a valores de una tupla

Se accede por **índice**, igual que con las listas:

```
frutas = ("manzana", "banana", "naranja")

print(frutas[0])    # manzana
print(frutas[1])    # banana
print(frutas[-1])   # naranja (último elemento)
```

También se puede hacer **slicing**:

```
print(frutas[0:2])  # ('manzana', 'banana')
print(frutas[::-1]) # ('naranja', 'banana', 'manzana')
```

Tuplas son inmutables — no se pueden modificar

Si intentás cambiar un elemento, Python lanza un error:

```
frutas = ("manzana", "banana", "naranja")  
frutas[0] = "pera"
```

```
TypeError: 'tuple' object does not support item assignment
```

Tampoco se pueden agregar ni eliminar elementos:

```
frutas.append("pera") # AttributeError  
frutas.remove("banana") # AttributeError
```

Eliminar una tupla completa

No se puede eliminar un **elemento** de una tupla, pero sí se puede **eliminar toda la tupla** con `del` :

```
frutas = ("manzana", "banana", "naranja")  
del frutas  
  
print(frutas)    # NameError: name 'frutas' is not defined
```

⚠ Después del `del`, la variable ya no existe en memoria.

Métodos de las tuplas: `count()` e `index()`

Las tuplas tienen solo **dos métodos** (porque son inmutables):

```
numeros = (1, 2, 3, 2, 4, 2)

# count() → cuántas veces aparece un valor
print(numeros.count(2))    # 3

# index() → en qué posición está un valor (primera ocurrencia)
print(numeros.index(3))    # 2
print(numeros.index(2))    # 1
```

Si `index()` no encuentra el valor, lanza un `ValueError`.

Iterar una tupla

Se puede recorrer una tupla igual que una lista:

```
frutas = ("manzana", "banana", "naranja")
```

```
# Con for simple  
for fruta in frutas:  
    print(fruta)
```

```
# Con enumerate (índice + valor)  
for i, fruta in enumerate(frutas):  
    print(f"{i}: {fruta}")
```

```
# Con while  
i = 0  
while i < len(frutas):  
    print(frutas[i])  
    i += 1
```


Tuple Unpacking (desempaquetar)

Se puede asignar cada elemento de una tupla a una variable distinta:

```
coordenadas = (40.7128, -74.0060)
latitud, longitud = coordenadas
```

```
print(latitud)      # 40.7128
print(longitud)     # -74.006
```

```
persona = ("Ana", 28, "Buenos Aires")
nombre, edad, ciudad = persona
```

```
print(f"{nombre} tiene {edad} años y vive en {ciudad}.")
# Ana tiene 28 años y vive en Buenos Aires.
```

⚠ La cantidad de variables debe coincidir con la cantidad de elementos.

Tuple Packing (empaquetar)

Se puede crear una tupla asignando múltiples valores **sin paréntesis**:

```
# Tuple packing
datos = "Juan", 25, "Argentina"
print(datos)          # ('Juan', 25, 'Argentina')
print(type(datos))    # <class 'tuple'>
```

Muy útil para devolver múltiples valores desde una función:

```
def min_max(lista):
    return min(lista), max(lista)    # devuelve una tupla

minimo, maximo = min_max([3, 1, 7, 2, 5])
print(minimo, maximo)    # 1 7
```

Concatenar y hacer slicing de tuplas

```
# Concatenación con +
a = (1, 2, 3)
b = (4, 5, 6)
c = a + b
print(c)    # (1, 2, 3, 4, 5, 6)

# Repetición con *
d = (0,) * 3
print(d)    # (0, 0, 0)

# Slicing
numeros = (0, 1, 2, 3, 4, 5)
print(numeros[1:4])    # (1, 2, 3)
print(numeros[::2])    # (0, 2, 4)
print(numeros[::-1])   # (5, 4, 3, 2, 1, 0)
```

Casos de uso de las tuplas

Las tuplas son ideales cuando los datos **no deben cambiar**:

```
# Coordenadas geográficas
buenos_aires = (-34.6037, -58.3816)
nueva_york = (40.7128, -74.0060)

# Días de la semana (datos fijos)
dias = ("lunes", "martes", "miércoles", "jueves",
        "viernes", "sábado", "domingo")

# Retornar múltiples valores
def dividir(a, b):
    cociente = a // b
    resto = a % b
    return cociente, resto

c, r = dividir(17, 5)
print(f"Cociente: {c}, Resto: {r}")    # Cociente: 3, Resto: 2
```

¿Qué es un Set (Conjunto)?

Un **set** es una colección **desordenada** de valores **únicos** (sin duplicados).

- **Desordenado:** los elementos no tienen índice ni posición fija
- **Sin duplicados:** si agregás un valor que ya existe, se ignora
- **Mutable:** se pueden agregar y eliminar elementos

```
numeros = {1, 2, 3, 2, 1}
print(numeros)    # {1, 2, 3} ← los duplicados se eliminan
```

Crear sets en Python

```
# Set con elementos
frutas = {"manzana", "banana", "naranja"}
print(frutas)    # {'banana', 'naranja', 'manzana'} (orden aleatorio)

# Set vacío → DEBE usarse set(), no {}
vacio = set()    # ✅ set vacío
no_vacio = {}    # ❌ esto crea un diccionario, no un set!

# Crear set desde una lista (elimina duplicados)
lista = [1, 2, 2, 3, 3, 3]
sin_duplicados = set(lista)
print(sin_duplicados)    # {1, 2, 3}
```

Acceder a valores — los sets no tienen índice

Los sets son **desordenados**, por lo que no se puede acceder por índice:

```
frutas = {"manzana", "banana", "naranja"}  
  
# Esto NO funciona:  
print(frutas[0])    # TypeError: 'set' object is not subscriptable
```

Para verificar si un elemento existe, se usa `in`:

```
print("manzana" in frutas)    # True  
print("pera" in frutas)      # False  
print("pera" not in frutas)   # True
```

Agregar y eliminar elementos

```
frutas = {"manzana", "banana"}

# Agregar un elemento
frutas.add("naranja")
print(frutas)    # {'manzana', 'banana', 'naranja'}

# Agregar uno que ya existe → no hace nada
frutas.add("manzana")
print(frutas)    # {'manzana', 'banana', 'naranja'}

# Eliminar con remove() → KeyError si no existe
frutas.remove("banana")

# Eliminar con discard() → no lanza error si no existe
frutas.discard("pera")    # no pasa nada
```


Más métodos de los sets

Método	Descripción
<code>add(x)</code>	Agrega <code>x</code> al set
<code>remove(x)</code>	Elimina <code>x</code> (error si no existe)
<code>discard(x)</code>	Elimina <code>x</code> (sin error si no existe)
<code>pop()</code>	Elimina y retorna un elemento aleatorio
<code>clear()</code>	Vacía el set
<code>copy()</code>	Retorna una copia del set

Unión de sets

La **unión** combina todos los elementos de ambos sets:

```
a = {1, 2, 3}
b = {3, 4, 5}

# Con el método union()
print(a.union(b))    # {1, 2, 3, 4, 5}

# Con el operador |
print(a | b)         # {1, 2, 3, 4, 5}
```

Los duplicados (como el 3) aparecen **una sola vez** en el resultado.

Intersección de sets

La **intersección** retorna solo los elementos que están en **ambos** sets:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Con el método intersection()
print(a.intersection(b))    # {3, 4}

# Con el operador &
print(a & b)                 # {3, 4}
```

Útil para encontrar elementos en común entre dos colecciones.

Diferencia de sets

La **diferencia** retorna los elementos que están en **a** pero **no** en **b** :

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Con el método difference()
print(a.difference(b))    # {1, 2}

# Con el operador -
print(a - b)              # {1, 2}

# Diferencia en sentido contrario
print(b - a)              # {5, 6}
```

Iterar un set con `for`

Se puede recorrer un set con `for`, aunque el **orden no está garantizado**:

```
frutas = {"manzana", "banana", "naranja"}  
  
for fruta in frutas:  
    print(fruta)  
# banana, naranja, manzana (en orden no garantizado)
```

(El orden puede variar cada vez que ejecutás el programa.)

Si necesitás orden, convertí el set a lista primero:

```
for fruta in sorted(frutas):  
    print(fruta)    # orden alfabético garantizado
```

Resumen: Tupla vs Set vs Lista

	Lista <code>[]</code>	Tupla <code>()</code>	Set <code>{}</code>
Ordenada	✓	✓	✗
Mutable	✓	✗	✓
Duplicados	✓	✓	✗
Índice	✓	✓	✗
Uso típico	Colección general	Datos fijos	Valores únicos

```
lista = [1, 2, 2, 3]      # mutable, permite duplicados
tupla = (1, 2, 2, 3)     # inmutable, permite duplicados
conjunto = {1, 2, 2, 3}  # mutable, sin duplicados → {1, 2, 3}
```

Conceptos clave de la clase

Tuplas:

- Colección ordenada e **inmutable** → `(elem1, elem2)`
- Solo tienen `count()` e `index()`
- **Unpacking:** `a, b = (1, 2)`
- Ideales para datos que no cambian (coordenadas, retorno múltiple)

Sets:

- Colección **desordenada** y **sin duplicados** → `{elem1, elem2}`
- Sin acceso por índice; usar `in` para verificar membresía
- Operaciones: unión (`|`), intersección (`&`), diferencia (`-`)
- Perfectos para eliminar duplicados y comparar colecciones